

Computational Design Thinking

Nature of Problems:
Well-Structured vs Ill-Structured
Problems

B.Tech CSE – 1st Year

Learning Objectives

Understand Computational Design Thinking.

Understand the nature of problems.

Differentiate well-structured and Ill-structured problems.

Identify real-world examples of both types.

Choose an appropriate problem-solving approach.

What is Computational Design Thinking?

A systematic way of understanding and solving problems using logical and computational thinking.

It combines problem understanding, decomposition, solution design and step-by-step representation.

Simple flow: Understand → Break Down → Think → Design → Represent → Evaluate.

The aim is to create a logical, practical and explainable solution.

What is a Problem?

A problem is a situation in which we have a goal, but the path to reach that goal is not immediately known.

Problem-solving starts by identifying the current situation, desired goal and constraints.

Different problems have different levels of clarity, information and possible solutions.

Nature of Problems

Some problems have a clear goal, known inputs, fixed rules and a predictable solution path.

Other problems may have incomplete information, multiple stakeholders, competing goals and many possible solutions.

Therefore, problems can broadly be understood as Well-Structured and Ill-Structured.

Well-Structured Problems

Goal is clearly defined.

Inputs and constraints are known.

Rules, formulas or procedures are available.

Usually has one definite or objectively verifiable answer.

Solution can often be represented as an algorithm.

Example: Calculate the percentage of five given marks.

III-Structured Problems

Goal may be unclear or open to interpretation.

Information may be incomplete or uncertain.

Multiple constraints and stakeholders may exist.

Several solutions may be acceptable.

Requires judgment, creativity, design thinking and evaluation.

Example: How can a college improve student engagement?

Well-Structured vs Ill-Structured

Well-Structured: Clear goal, known data, fixed rules, definite answer, algorithm-friendly.

Ill-Structured: Ambiguous goal, incomplete data, changing constraints, multiple acceptable answers.

Well-structured problems emphasize accuracy and efficiency.

Ill-structured problems emphasize creativity, judgment, trade-offs and iteration.

Well-Structured vs Ill-Structured Problems

Well-Structured: Clear goal, known inputs and constraints, defined rules/procedures, and usually a definite or objectively verifiable answer.

Ill-Structured: Goal may be ambiguous, information may be incomplete, constraints may change, and multiple acceptable solutions may exist.

Well-Structured problems are generally suitable for systematic algorithms.

Ill-Structured problems require judgment, creativity, design thinking and evaluation.

Approaching an Ill-Structured Problem

1. Understand the context and stakeholders.
2. Define or reframe the problem clearly.
3. Identify constraints and missing information.
4. Generate multiple possible solutions.
5. Compare solutions using suitable criteria.
6. Prototype, evaluate and improve using feedback.

Role of Computational Design Thinking

For well-structured problems: focus on accuracy, efficiency, algorithms and clear procedures.

For ill-structured problems: organize uncertainty, decompose complexity and compare alternatives.

It connects human-centered design with logical and computational problem solving.

Good problem solving starts with the right problem definition.

Key Takeaways

Computational Design Thinking = systematic problem understanding + logical solution design.

Understand the problem before designing the solution.

Well-structured problems have clear goals, inputs and rules.

Ill-structured problems involve ambiguity, uncertainty and multiple possible solutions.

The nature of the problem determines the problem-solving strategy.

Well-Structured vs Ill-Structured Problems

Well-Structured: Clear goal, known inputs and constraints, defined rules/procedures, and usually a definite or objectively verifiable answer.

Ill-Structured: Goal may be ambiguous, information may be incomplete, constraints may change, and multiple acceptable solutions may exist.

Well-Structured problems are generally suitable for systematic algorithms.

Ill-Structured problems require judgment, creativity, design thinking and evaluation.